

Programación declarativa

- Haskell •

Doble grado en Ingeniería Informática y Matemáticas

Universidad Complutense de Madrid

Óscar Martín — Curso 2026/27

15. Definición de tipos

Nombrar tipos existentes: `type`

```
type Nombre = String
type Apellido = String
dni :: Nombre -> Apellido -> Int
dni _ _ = 0

type Punto2D = (Float, Float)
type Complejo = (Float, Float)

sumaComplejo :: Complejo -> Complejo -> Complejo
sumaComplejo (r1, i1) (r2, i2) = (r1+r2, i1+i2)

punto, otro :: Punto2D
punto = (1, 1)
otro = sumaComplejo punto punto

type String = [Char]

type Diccionario k v = [(k, v)]

dic :: Diccionario Int String
dic = [(1, "uno"), (2, "dos"), (3, "tres")]

busca :: Eq k => k -> Diccionario k v -> [v]
busca k d = [v | (k', v) <- d, k == k']

valor :: [String]
valor = busca 3 dic -- ["tres"]
```

Crear nuevos tipos: data

```
| data Dia = Lu | Ma | Mi | Ju | Vi | Sa | Do
```

- `Dia` es el nombre del nuevo tipo.
- `Lu`, `Ma`, etc. son los valores que tienen ese tipo.

```
– :t Lu ~> Lu :: Dia
```

```
| esLaborable :: Dia -> Bool
```

```
| esLaborable Sa = False
```

```
| esLaborable Do = False
```

```
| esLaborable _ = True
```

```
| data Figura = Circulo Punto2D Float
```

```
| Rectangulo Punto2D Float Float
```

```
| area :: Figura -> Float
```

```
| area (Circulo _ r) = pi * r^2
```

```
| area (Rectangulo _ x y) = x * y
```

- `Circulo` y `Rectangulo` son constructores de valores.
- Los nombres de tipos y de constructores tienen que empezar con mayúscula.
- Los constructores son como funciones:

```
– :t Circulo ~> Circulo :: Punto2D -> Float -> Figura
```

- Incluso se pueden *currifcar*:

```
– map area (map (Circulo (10, 20)) [4, 5, 6, 11])
```

- Pero, a diferencia de las funciones, los constructores no tienen definición:

```
– Circulo (0, 0) 1 no se evalúa, ya es un valor.
```

- Y las funciones no se pueden usar en patrones; los constructores, sí.
- Los tipos también pueden tener argumentos, que son otros tipos:

```
| data Intervalo a = Intervalo a a
```

- El `Intervalo` de la izquierda es un *constructor de tipos*:

```
– Intervalo Float es un tipo.
```

- El `Intervalo` de la derecha es un *constructor de valores*:

```
– Intervalo 2.0 3.1 es un valor.
```

- Cuando hay un solo constructor, es frecuente dar el mismo nombre al constructor de tipos que al constructor de valores.

- Los espacios de nombres para tipos y para valores están separados en Haskell.

```

contiene :: Ord a => Intervalo a -> a -> Bool
contiene (Intervalo x y) z = x <= z && z <= y

solapan :: Ord a => Intervalo a -> Intervalo a -> Bool
solapan (Intervalo a b) (Intervalo c d) = a <= d && c <= b

```

- `contiene (Intervalo 'a' 'z') 'Q' ~> False`
- `solapan (Intervalo 1.5 3.0) (Intervalo 2.5 4.0) ~> True`
- `:t Intervalo 'a' 'z' ~> Intervalo 'a' 'z' :: Intervalo Char`
- **newtype** es lo mismo que **data** pero solo admite un constructor con un campo.
 - A cambio, tiene mejor coste de ejecución.

Either y Maybe

- El constructor de tipos **Either** está definido así en Haskell:

```

data Either a b = Left a | Right b

type EnteroOLetra = Either Int Char

sumaEnteros :: [EnteroOLetra] -> Int
sumaEnteros [] = 0
sumaEnteros (Left x : xs) = x + sumaEnteros xs
sumaEnteros (Right _ : xs) = sumaEnteros xs

```

- Podemos preferir dar nombres nuestros a los constructores:

```

data EnteroOLetra' = Entero Int | Letra Char

sumaEnteros' :: [EnteroOLetra'] -> Int
sumaEnteros' [] = 0
sumaEnteros' (Entero x : xs) = x + sumaEnteros' xs
sumaEnteros' (Letra _ : xs) = sumaEnteros' xs

```

- El constructor de tipos **Maybe** está definido así en Haskell:

```

data Maybe a = Nothing | Just a

divSegura :: Int -> Int -> Maybe Int
divSegura _ 0 = Nothing
divSegura m n = Just (div m n)

```

Tipos recursivos

```
| data Nat = Cero | Suc Nat
```

- Los valores de tipo `Nat` son `Cero`, `Suc Cero`, `Suc (Suc Cero)`, ...

```
| fromNat :: Num a => Nat -> a
```

```
| fromNat Cero = 0
```

```
| fromNat (Suc n) = 1 + fromNat n
```

```
| data Lista a = Vacia | Cons a (Lista a)
```

```
| longitud :: Lista a -> Int
```

```
| longitud Vacia = 0
```

```
| longitud (Cons _ xs) = 1 + longitud xs
```

```
| data Arbol a = Nodo a (Bosque a)
```

```
| type Bosque a = [Arbol a]
```

Ejercicios

1. Define una función `map` que opere sobre elementos de tipo `Lista a`. (Si quieres que `ghci` muestre valores de tu tipo `Lista`, añade `deriving Show` al final de la declaración `data`; se explica más adelante.)
2. Define una función `sumaArbol :: Num a => Arbol a -> a` que sume todos los nodos de un `Arbol a`. Usa para ello `sumaBosque`, que también debes definir.
3. Define un tipo de árboles binarios, es decir, árboles en los que cada nodo puede tener un máximo de dos subárboles. Por tanto, un árbol es, o bien vacío, o bien un nodo que guarda un dato y tiene dos subárboles (y cualquiera de ellos puede ser vacío). Define una función `numHojas` que diga cuántas hojas tiene un árbol binario. Define también una función `mapAB` para estos árboles. (Si quieres que `ghci` muestre valores de tu tipo, añade `deriving Show` al final de la declaración `data`; se explica más adelante.)